SUP DE VINCI RENNES

MASTÈRE EXPERT EN DÉVELOPPEMENT ET DÉPLOIEMENT DE SOLUTIONS NUMÉRIQUES SÉCURISÉES

M2 · Session 2026–2027 · RNCP 40166

CAHIER DES CHARGES

PROJET FIL ROUGE CI/CD

SOUVERAINRECRUT

Industrialisation, sécurisation et déploiement
d'une API Backend

Module	CI/CD – 14 heures
Durée	Deux journées de 7 heures
Filière	Mastère Développement
Formateur	Alimou Diallo
Compétences visées	RNCP 40166 – Blocs 4 et 5

Finalité du projet
Mettre en œuvre une chaîne complète et reproductible, depuis le contrôle du code source jusqu'au déploiement automatisé d'une API Backend sur un serveur.

Dépôt de référence : github.com/Santoudllo/souverainrecrut

1. CONTEXTE ET FINALITÉ

Dans le cadre du module CI/CD, vous devez industrialiser le cycle de vie de SouverainRecrut, une application de gestion de recrutement. Le projet sert de support à la mise en œuvre des pratiques d'intégration continue, de contrôle qualité, de conteneurisation, de sécurisation de la chaîne logicielle et de déploiement automatisé.

Périmètre retenu : Backend uniquement. Afin de concentrer le travail sur les pratiques CI/CD et de limiter la charge du projet, le Frontend React est exclu du périmètre. Les travaux portent exclusivement sur l'API Backend et sa base de données PostgreSQL.

Composants concernés

• **Backend :** API REST développée avec Node.js 22+, Express, TypeScript et Prisma ORM.

• **Base de données :** PostgreSQL, utilisée par l'API via Prisma.

Dépôt GitHub de référence : https://github.com/Santoudllo/souverainrecrut

Objectif général

Construire une chaîne de livraison dans laquelle le code Backend est testé et analysé, une image Docker est construite, analysée et publiée sur Docker Hub, puis récupérée par un pipeline GitLab CI/CD afin d'être déployée sur un serveur Ubuntu préparé avec Ansible et, selon l'environnement disponible, Terraform.

2. PÉRIMÈTRE TECHNIQUE ET ARCHITECTURE CIBLE

CHAÎNE D'INTÉGRATION ET DE PUBLICATION	CHAÎNE DE PROVISIONNEMENT ET DE DÉPLOIEMENT
GitHub – branche Dev ↓ Jenkins ↓ Tests automatisés Backend ↓ SonarQube / Quality Gate ↓ Build Docker + scan Trivy ↓ Docker Hub **Image Backend versionnée**	VMware / Ubuntu Server ↓ Terraform (selon environnement) ↓ Ansible ↓ GitLab CI/CD ↓ Pull de l'image Docker Hub ↓ Docker Compose + PostgreSQL **API déployée**

Outils et technologies

Domaine	Technologies
Code et dépôts	Git, GitHub, GitLab
Intégration et qualité	Jenkins, SonarQube, tests automatisés
Conteneurs et sécurité	Docker, Docker Compose, Trivy, Docker Hub
Infrastructure	VMware, Ubuntu Server, Terraform, Ansible
Déploiement	GitLab CI/CD, SSH, PostgreSQL, Prisma

Le projet porte exclusivement sur le Backend. Aucune tâche de développement, de test, de construction ou de déploiement du Frontend n'est attendue.

3. TRAVAUX À RÉALISER – INTÉGRATION CONTINUE AVEC JENKINS

Durée indicative : 7 heures. À l'issue de cette partie, l'API Backend doit être testée, analysée, conteneurisée et publiée sur Docker Hub.

3.1. Préparation du dépôt

• Disposer de Git, Node.js 22+ et Docker, ainsi que de comptes GitHub et Docker Hub.

• Créer un fork personnel du dépôt officiel SouverainRecrut.

• Cloner votre fork et vous positionner sur la branche de travail Dev.

```
git clone https://github.com/VOTRE_UTILISATEUR/souverainrecrut.git
cd souverainrecrut
git checkout Dev
cd backend
npm ci
npx prisma generate
```

3.2. Mise en place des tests Backend

Le dépôt fourni constitue un point de départ : les scripts de test présents dans le Backend sont des placeholders et aucun framework de test complet n'est préconfiguré. Vous devez donc choisir et configurer un environnement de tests adapté à l'API, puis créer des tests unitaires et/ou d'intégration.

• Configurer les dépendances, scripts npm et paramètres de couverture.

• Écrire des tests couvrant les principales fonctionnalités de l'API : authentification, gestion des candidats, offres, candidatures et contrôle des accès.

• Produire un rapport de couverture exploitable par la pipeline et SonarQube.

3.3. Analyse de la qualité avec SonarQube

• Configurer le projet et les paramètres d'analyse SonarQube.

• Examiner les bugs, vulnérabilités, code smells, security hotspots, duplications et couverture.

• Configurer et vérifier le Quality Gate. L'objectif pédagogique est d'obtenir un statut PASSED selon les règles définies pour le projet.

3.4. Mise en place de Jenkins

• Configurer Jenkins et l'accès au dépôt GitHub, puis activer les plugins Git et Pipeline ainsi que l'intégration SonarQube.

• Configurer les credentials et l'accès à Docker sans inscrire de secret dans le dépôt.

• Créer un Jenkinsfile versionné à la racine du projet ou dans le répertoire convenu.

3.5. Automatisation de la chaîne CI

Le Jenkinsfile doit organiser les étapes suivantes : checkout de la branche Dev, installation des dépendances, génération Prisma, exécution des tests, analyse SonarQube et contrôle du Quality Gate, construction de l'image Backend, scan Trivy, puis publication conditionnée à la réussite des contrôles.

3.6. Construction et publication de l’image Docker

Le dépôt fourni contient déjà un Dockerfile Backend. Vous devez le vérifier et l’adapter si nécessaire, notamment pour obtenir une image reproductible et ne pas embarquer les fichiers ou secrets inutiles.

• Construire une image Docker du Backend.

• Utiliser des tags explicites et traçables : version applicative et/ou identifiant de commit.

• Analyser l’image avec Trivy et définir les seuils de vulnérabilités acceptés.

• Publier l’image sur Docker Hub uniquement après validation des contrôles retenus.

```
docker build -t VOTRE_COMPTE/souverainrecrut-backend:VERSION .
docker image ls
docker push VOTRE_COMPTE/souverainrecrut-backend:VERSION
```

Résultat attendu

Un Jenkinsfile opérationnel, des tests Backend exécutables, un rapport de couverture, les résultats SonarQube et Trivy ainsi qu’une image Backend versionnée publiée sur Docker Hub.

4. TRAVAUX À RÉALISER – PROVISIONNEMENT ET DÉPLOIEMENT

Durée indicative : 7 heures. À l’issue de cette partie, l’API Backend doit être accessible sur un serveur Ubuntu et communiquer avec PostgreSQL.

4.1. Préparation de l’environnement VMware

Installer VMware Workstation ou utiliser l’environnement VMware mis à disposition. Créer une machine virtuelle dédiée au déploiement avec la configuration indicative suivante :

Ressource	Configuration indicative
Système	Ubuntu Server 24.04 LTS
Processeur	2 vCPU minimum
Mémoire	4 Go RAM minimum
Stockage	30 Go minimum
Réseau	NAT ou Bridged, avec accès Internet
Administration	SSH activé et accès distant vérifié

• Installer et mettre à jour Ubuntu Server.

• Vérifier l’adresse IP, l’accès Internet et la connexion SSH.

• Réaliser un snapshot avant les opérations de configuration.

4.2. Provisionnement avec Terraform

Créer un répertoire terraform/ et organiser les fichiers main.tf, variables.tf, outputs.tf et providers.tf. Décrire les ressources nécessaires selon les capacités de l’environnement VMware disponible.

```
terraform init
terraform validate
terraform plan
terraform apply
```

Le provider Terraform vSphere nécessite un environnement ESXi/vCenter compatible. VMware Workstation seul ne permet pas de créer directement une VM avec ce provider. Sans ESXi/vCenter, la création initiale de la VM est manuelle ; Terraform peut être utilisé sur les ressources réellement disponibles.

4.3. Configuration du serveur avec Ansible

Créer un inventaire SSH et un playbook Ansible afin d'automatiser la préparation du serveur. Le playbook doit notamment :

• Mettre à jour le système et installer les dépendances nécessaires.

• Installer Docker Engine et le plugin Docker Compose.

• Activer et démarrer le service Docker.

• Créer le répertoire de déploiement et vérifier le fonctionnement des services.

```
ansible-playbook -i inventory.ini playbook.yml
```

4.4. Préparation du projet GitLab

Créer un projet GitLab dédié au déploiement et préparer le fichier .gitlab-ci.yml. Configurer les variables CI/CD nécessaires : identifiants et token Docker Hub, adresse du serveur, utilisateur SSH, clé privée de déploiement et tag de l'image Backend à déployer.

Les secrets doivent être enregistrés dans les variables sécurisées de GitLab. Ils ne doivent jamais être ajoutés au dépôt ni affichés dans les logs.

5. DÉPLOIEMENT APPLICATIF ET VALIDATION

5.1. Récupération de l'image Docker Hub

Le serveur distant doit récupérer l'image Backend publiée par Jenkins. Le pipeline de déploiement ne doit pas reconstruire l'image depuis le code source.

```
docker login
docker pull VOTRE_COMPTE/souverainrecrut-backend:VERSION
```

5.2. Déploiement avec Docker Compose

Créer un répertoire deployment/ contenant un fichier docker-compose.yml et un modèle .env.example. Le fichier Compose doit orchestrer les services Backend et PostgreSQL.

• Utiliser l'image Backend versionnée publiée sur Docker Hub.

• Définir les variables d'environnement nécessaires à l'API et à Prisma.

• Prévoir un volume persistant pour les données PostgreSQL et un réseau Docker adapté.

• Prévoir des politiques de redémarrage et des contrôles de santé.

• Appliquer les migrations Prisma nécessaires selon une stratégie maîtrisée.

```
docker compose pull
docker compose up -d
docker compose ps
docker compose logs -f
```

5.3. Pipeline de déploiement GitLab CI/CD

Le pipeline GitLab doit valider le tag de l'image, se connecter au serveur en SSH, transmettre les fichiers de déploiement et la configuration nécessaire, récupérer l'image Docker Hub, puis démarrer ou mettre à jour les services. Il doit enfin vérifier l'état des conteneurs et la disponibilité de l'API.

Prévoir une approbation manuelle avant le déploiement en production, conformément au processus de validation retenu.

5.4. Vérification et procédure de rollback

• Vérifier l'accessibilité de l'API et le bon fonctionnement de ses principaux endpoints.

• Vérifier la communication entre le Backend et PostgreSQL ainsi que la persistance des données.

• Identifier une version précédente de l'image et documenter le retour à cette version.

• Tester la procédure de rollback sur un scénario maîtrisé.

Le rollback applicatif ne garantit pas automatiquement la réversibilité des changements de schéma de base de données. Les migrations Prisma doivent être conçues en tenant compte de ce risque.

Résultat attendu

Une API Backend accessible sur le serveur, déployée à partir d'une image Docker Hub identifiée, avec une procédure de vérification et de retour arrière documentée.

6. LIVRABLES À REMETTRE

Le dépôt Git ou l'archive ZIP doit contenir les fichiers nécessaires à la compréhension et à la reproduction du projet. Organisation indicative :

```
SouverainRecrut-CICD/
■■■ integration-continue/
■   ■■■ Jenkinsfile
■   ■■■ Dockerfile
■   ■■■ sonar/
■   ■■■ reports/
■   ■■■ README.md
■■■ deploiement/
■   ■■■ terraform/
■   ■■■ ansible/
■   ■■■ deployment/
■   ■   ■■■ docker-compose.yml
■   ■   ■■■ .env.example
■   ■■■ .gitlab-ci.yml
■   ■■■ README.md
■■■ documentation/
    ■■■ architecture.md
    ■■■ procedure-deploiement.md
    ■■■ procedure-rollback.md
    ■■■ rapport-final.md
```

Joindre également les rapports de tests et de couverture, les résultats SonarQube et Trivy ainsi que des captures d'écran des principales étapes. Les fichiers contenant des secrets, mots de passe, tokens, fichiers .env réels et clés privées doivent être exclus du livrable.

7. EXIGENCES DE QUALITÉ ET DE SÉCURITÉ

Exigence	Attendu
Reproductibilité	Les étapes de build, de configuration et de déploiement sont décrites et rejouables.
Qualité logicielle	Les tests Backend sont exécutés ; les résultats de couverture et SonarQube sont consultables.
Sécurité	Les secrets sont protégés ; l'image Backend est analysée avec Trivy.
Traçabilité	L'image est identifiée par un tag explicite et reliée à une version du code.
Déploiement	Le serveur récupère l'image publiée et l'API démarre avec PostgreSQL.
Continuité	Une procédure de rollback vers une version antérieure est documentée.
Documentation	Les choix techniques, prérequis et procédures sont expliqués.

8. ORGANISATION ET ATTENDUS PÉDAGOGIQUES

Le projet est réalisé sur deux journées de formation de 7 heures. Il mobilise les compétences d'industrialisation de la construction logicielle, d'automatisation des contrôles qualité, de sécurisation de la chaîne de livraison et de déploiement sur infrastructure.

Chaque étudiant doit être en mesure de présenter l’architecture mise en œuvre, d’expliquer le rôle des outils utilisés, de justifier les choix techniques et de démontrer le cheminement complet entre une modification du code Backend et la version déployée de l’API.

Le travail attendu ne se limite pas à obtenir une API fonctionnelle. Il consiste à produire un processus de livraison automatisé, reproductible, sécurisé, traçable et documenté.

Modalités de remise

Déposer une archive ZIP contenant les fichiers de configuration, les rapports et la documentation, ou transmettre le lien du dépôt GitHub / GitLab regroupant l’ensemble des livrables. Le dépôt doit être accessible et ne contenir aucun secret.